Devops & Cloud Engineering With AI - Notes By Vivek Taware

Step -- 01:-

1) Why we use linux in AWS:-it provides a fast, stable and free foundation that handles high – speed cloud Computing without expensive licensing fees
2) Git bash or Ubuntu → for practice linux commands or use linux operating system
3) Basic Commands of Linux
   - touch vivek.txt → for create file vivek.txt
   - ls → lists all files and directories
   - echo → for write in one line text eg : echo “hallo” vivek.txt
   - cat vivek.txt → print or view file’s contents
   - vi vivek.txt → for Write too much line’s or edit whole file → :x – to save :wq! – to exit
   - :set nu – for → in vi editor we this for assign line number’s
   - mkdir → used to create directory
   - pwd → show the user or current user
   - cd → used to change directory
   - cd .. → used to back or exit from directory
   - cp → cp vivek1.txt vivek2.txt → used to copy file and paste it
   - mv → mv vivek1.txt vivek2.txt folder1/ /path/to/destination_folder/ → used to move files
   - head -n → using this command we print first lines of every file
   - tail -n → using this command we print ending of every file
   - whoami → show current user
   - top → is used to monitor running processor and system resources usage in real time
   - df → is used to check disk space usage of filesystem
   - df -h → for a human readable format output
   - ls -l | grep vivek.txt → this command is used like sort or find the files using file name
   - ssh -keygen → used to create ssh keys it creates a pair of keys
   - cd ~/.ssh → used to go inside the ssh directory
   - scp -i “keypair name” “file name” ubuntu@public_ip_secondEC2:/home/ubuntu/ → to access another server or to catch file from another server

   - **AWS CLI**
   - aws --version → Check AWS CLI version
   - aws configure →Configure AWS CLI
   - aws configure list →Show AWS CLI configuration
   - aws sts get-caller-identity →Show current AWS identity

VIVEK TAWARE 9322662939

- **EC2**
- aws ec2 describe-instances → List EC2 instances
- aws ec2 start-instances --instance-ids INSTANCE_ID →Start EC2 instance
- aws ec2 stop-instances --instance-ids INSTANCE_ID →Stop EC2 instance
- aws ec2 reboot-instances --instance-ids INSTANCE_ID →Reboot EC2 instance

- **S3**
- aws s3 ls →List S3 buckets
- aws s3 ls s3://BUCKET_NAME →List bucket contents
- aws s3 mb s3://BUCKET_NAME →Create bucket
- aws s3 cp file.txt s3://BUCKET_NAME/ →Upload file
- aws s3 cp s3://BUCKET_NAME/file.txt . →Download file
- aws s3 sync ./folder s3://BUCKET_NAME/ →Sync folder to S3
- aws s3 rm s3://BUCKET_NAME/file.txt →Delete S3 object

VIVEK TAWARE 9322662939

- **VPC**
- aws ec2 describe-vpcs → List VPCs
- aws ec2 describe-subnets →List subnets
- aws ec2 describe-route-tables →List route tables
- aws ec2 describe-internet-gateways →List Internet Gateways
- aws ec2 describe-nat-gateways →List NAT Gateways
- aws ec2 describe-security-groups →List security groups

- **IAM**
- aws iam list-users →List IAM users
- aws iam list-roles →List IAM roles
- aws iam list-policies →List IAM policies

Step –2: -

AWS Account Opening → open free user account to practice

1) what we learn in AWS -- Amazon Web Services

1. Networking
2. Computing – EC2
3. Storage
4. Autoscale
5. Lambda
6. RDS – Relational Database system
7. IAM user

2) Networking

1. VPC
2. EC2
3. EBS
4. EFS
5. Aurora and RDS
6. IAM
7. Dynamo DB

1) Networking
    1. What is VPC
        - VPC stands for Virtual Private Cloud which is a secure, isolated and private network hosted within a public cloud
    2. Subnet
        - Subnet is used to organize and secure your cloud resource
        - Using subnet, we divide a big IP into a small IP set
        - Also used to represent the private internet in EC2 instance
    3. Route-Table
        - A digital map or list of rules inside a router or cloud network that matches destination IP addresses to specific network path
    4. Use – path selection, Traffic control in cloud (VPC), Network Security, Handling unmatched Traffic
    5. What is Internet Gateway?
        - An internet Gateway is a hardware or software bridge that connect a local or private network to the public internet allowing data to flow between them
        - Def – a network node or components that acts as an entrance and exit point for traffic moving between an internal network (like a home network office LAN or cloud virtual private cloud)

VIVEK TAWARE 9322662939

    6. NAT Gateway
        - A NAT Gateway (Networking Address Translation) gateway is a managed cloud service that allows resources

          Inside a private network or subnet to connect outward to the public internet while blocking any unsolicited inbound connection from the outside

- **Practical: - of Networking by Vivek Taware – VPC, Subnet, Route-Table, Internet Gateway, NAT-Gateway, Peering Connection Endpoint**

1) VPC create Steps

1. Open AWS
2. Search VPC in Search bar
3. Click on VPC
4. Click on create VPC
5. Enter Name any To your VPC
6. Enter IP address Range For Reference – 192.168.0.0/16
7. Scroll down and create VPC

2) Create Subnet

1. Create subnet – click on create subnet
2. While creating subnet – select your VPC
3. Scroll Down – Enter Name for subnet
4. Choose Availability Zone – as per need if don't need then don't select keep as it is
5. Enter subnet IPv4 subnet CIRD block
   – enter the IP starting first two block match with your VPC IP 192.168.10.0/24 for subnet first the IP Should Look Like This The 24 is your traffic IP -256 ips
6. Scroll down and click on create subnet

3) Create Route Table

1. Click on route table
2. Click on create route table
3. Enter name for your route table
4. Select your VPC
5. And click on create route table

4) After creating Route table

1. Click on the check box with confirmation of your Route table name
2. Scroll down and you see subnet association
3. Click on subnet association
4. Click on edit subnet association
5. And you see your subnet that you created
6. Select that subnet and click on save

5) Create internet gateway

1. Click on internet gateway
2. Click on create internet gateway
3. Enter name for your internet gateway
4. Scroll down and click on create internet gateway
5. After creating internet gateway go one step back
6. Click on checkbox of your internet gateway
7. Go to actions button and click on action's
8. And attach the internet gateway with your VPC

6) If you want a private server then you need to create a "NAT gateway"

1. Click on NAT Gateway
2. Click on create NAT Gateway
3. Enter name for your NAT gateway

VIVEK TAWARE 9322662939

4. Select your VPC
5. Note: - if cannot see your VPC First
   - You need to check your internet Gateway is attached with your VPC or not
6. If it's attached you see's your VPC in NAT Gateway creating section
7. Scroll down and click on the create NAT Gateway
8. After Creating NAT Gateway That NAT Gateway attach or add into your Private Route Table that created for Private Server

Step 1 — Create VPC-1

Go to: AWS Console → VPC → Your VPCs → Create VPC

Name : VPC-1 IPv4 CIDR : - 192.168.0.0/16

Step 2 — Create Subnet in VPC-1

Go to: **VPC → Subnets → Create subnet**

Select VPC: VPC-1

Subnet name: Subnet-1 Availability Zone: ap-south-1a IPv4 CIDR: 192.168.1.0/24

VIVEK TAWARE 9322662939

Step 3 — Route Table

Go to: **VPC → Route Tables → Create route table**

Name: Public-Route-VPC1 Select VPC :- VPC-1

Subnet associations → Edit subnet associations

Select Subnet : Subnet-1

Route→ Edit routes → Add route

Add: Destination: 0.0.0.0/0 Target: Internet Gateway Select Internet Gateway Name

Step 4 — Create Internet Gateway for VPC-1

Go to: **VPC → Internet Gateways → Create internet gateway**

Name: Net-VPC1

Then: **Actions → Attach to VPC**

Select: VPC-1

Step 4 — Create NAT Gateway for Private Internet

Go to: **VPC → NAT Gateways → Create NAT gateway**

Route→ Edit routes → Add route

Add: Destination: 0.0.0.0/0 Target: NAT Gateway Select NAT Gateway Name

1) NGINX Server Connect ion AND User Data Practical

1. Step 1 using public VPC
2. Create a EC2 instance for perform the nginx practical
3. While creating EC2 instance
4. In Advances details scroll down
5. And you see's user data – option
6. In that option you can paste these commands
   - #bin/bash
   - sudo apt update → #for update packages
   - sudo apt install nginx -y → # for install Nginx
   - sudo systemctl start nginx → # for start Nginx
   - sudo systemctl enable nginx → # Enable Nginx after reboot
   - sudo systemctl status nginx → # see status of Nginx is active or not
   - echo "<h1>hello from Vivek in Nginx Server How can I help you </h1>">/var/www/html/index.html → #create simple webpage
7. this is first method to do
8. and second method is you can run this cammands manually in EC2 console
9. And third method is creating a .sh file and paste all commands in that file
   - Create file → vi vivek.sh
   - Paste all command in that file
   - Give permission to that file → chmod 500 vivek.sh
   - And execute that file with → ./vivek.sh
10. This is third method to run nginx server
11. ./filename.sh run this file using this command and you will see the output
12. Using your public IP paste the public IP on browser and you see your website is working

2) Endpoints --

1. First Create public and private Instances and public and private connectivity

- Then connect the private server to public server
- Search S3 in search bar
- Click on create bucket
- Enter name for your bucket
- Scroll down and click on create bucket

2. Install AWS CLI

- To install follow below commands
- sudo apt update
- sudo apt upgrade -y
- curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
- sudo apt install unzip -y
- unzip awscliv2.zip
- sudo ./aws/install
- aws configure
- aws s3 ls

VIVEK TAWARE 9322662939

3. **While you configure aws**
   - aws configure
   - that click on your AWS ID and your name is available
   - in right side top of the corner
   - and click on security credential (open new tab for that)
   - and after opening security credential
   - scroll down and click on create access key
   - that access key provides your AWS access to the your EC2 server
   - select AWS CLI command line interface
   - and click on create access key
   - Note: - if create access key then download your Excel file of access key's
4. **if you visible your bucket that's mean you have successfully installed AWS CLI**
   - now you delete your NAT Gateway and create endpoint
   - Enter name for your endpoint
   - Select a category (like AWS services – for s3 bucket access, and many)
   - Scroll down and search s3 and select s3 gateway same as your region

- com.amazonaws.us-east-1.s3	- amazon	- Gateway

   - In network setting select your VPC
   - Scroll down and select your Route table that created for private instance
   - Give full access and scroll down and click on create endpoint
   - After that all you run this command → aws s3 ls
   - To check your bucket is still available

3. Create EBS Volume → Attach to EC2 →

1. Search EC2 in search bar open EC2
2. scroll down and you see's Elastic Block Store EBS
3. in EBS click on volume's
4. click on create volume

- while creating this you need to know about the Volume types are available in volume setting while trying to create volume

Term	Simple meaning	Example
IOPS	How many input/output operations the disk can perform per second	Reading/writing many small database records
Throughput	How much data can be transferred per second	Reading a large 10 GB file
Latency	How quickly the disk responds to a request	Database query response

- types of volumes

- General Purpose SSD (gp3) → gp3 = modern, balanced SSD This is usually the best default choice for general EC2 workloads.
- General Purpose SSD (gp2) → gp2 = older generation general-purpose SSD It was widely used before gp3 became available.

VIVEK TAWARE 9322662939

- Provisioned IOPS SSD(io1) → io1 = high-performance SSD designed for I/O-intensive workloads. Here, you specifically provision the number of IOPS you need.
- Provisioned IOPS SSD(io2) → io2 = newer-generation provisioned IOPS SSD designed for very demanding workloads.
- Cold HDD (sc1) → **sc1 = low-cost HDD designed for infrequently accessed data.**
- Throughput Optimized HDD(st1) → st1 = HDD designed for high sequential throughput.
- Magnetic (Standard) → This is the **old magnetic EBS volume type**. It is a previous-generation HDD-based option.

5. Select size GiB in that range ---- Min: 1 GiB, Max: 65536 GiB.
6. Select availability zone
7. Scroll down and click on create volume button
8. After your volume is created click on that check box of your volume
9. And click actions
10. You see the "attach volume" option click that attach volume option
11. When attaching your volume with your EC2
12. After clicking you see's your volume ID
13. And check your availability zone
14. And select your instance
15. Most important point is device name (it's need to match device name with your actual volume device)
16. And then click on the attach volume

Steps For Attach and Mount EBS Volume to EC2

- Step 1 – Create EC2 instance
- Step 2 — Create an EBS Volume Go to AWS Console:
- EC2 → Elastic Block Store → Volumes → Create volume
- Setting → Value
- Volume type → gp3
- Size → 10 GiB
- Availability Zone → Same AZ as your EC2
- Tags → Name = EBS-Practical

***Important The EBS volume and EC2 instance must be in the same Availability Zone for normal attachment.

- For example:
- EC2 → ap-south-1a
- EBS → ap-south-1a
- Step 3 — Attach EBS to EC2
- Go to: EC2 → Volumes
- Select your new volume.
- Click: Actions → Attach volume Select your EC2 instance.

4. Format – Mount and Use EBS Volume

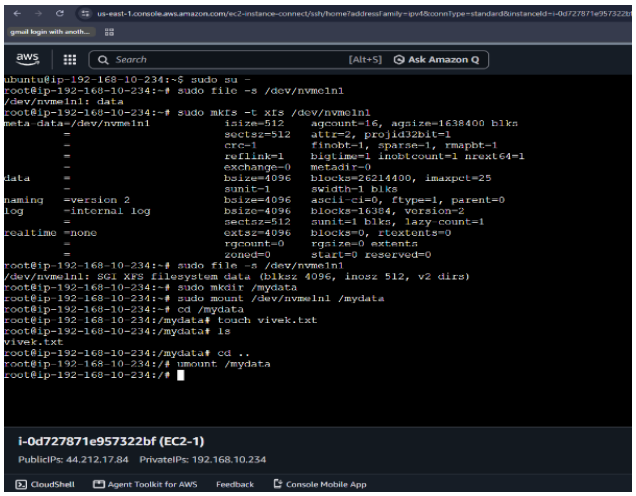
- After Creating EC2 and attached Volume to That EC2

- Follow Below Steps for Mount Volume and Use

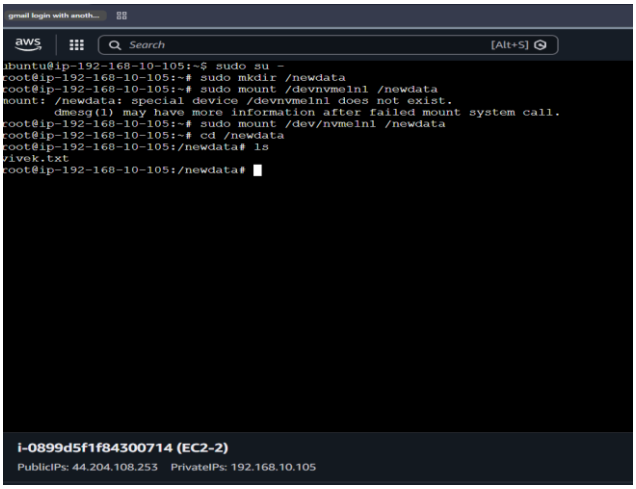
- and umount and attach That Volume second EC2 and Mount that volume you see's your Data as it is

VIVEK TAWARE 9322662939

➢ First Instance Work EC2-1



Second instance Work EC2-2



They Both Instance Have Same Availability Zone While We Diong This Practical Here're Below Steps and Command's To do This Practical in Right Way…

- sudo su - ---- Go to Root User To perform this practical
- lsblk ---- Check Volume – Displays all available storage devices / checking mount points.
- sudo fdisk -l ---- inspect disks – view create resize or delete disk partition tables on storage devices.
- sudo file -s /dev/nvme1n1 ---- check file system – inside a storage device to detect its filesystem or data structure without mounting it.
- sudo mkfs -t xfs /dev/nvme1n1 ---- if new / raw volume -- Formats A storage Device by building an XFS file system on it making it ready to store files.
- sudo file -s /dev/nvme1n1 ---- Verify Filesystem – safely inspects a storage device to detect its underlying filesystem type or partition data without mounting it.
- sudo mkdir /mydata ---- Create Mount Point – Create directory for mounting data in that specific directory.
- sudo mount /dev/nvme1n1 /mydata ---- This command is used for mount you're Directory With that volume you created and store data in that created volume.
- cd /mydata ---- go back from that directory.
- umount /mydata ---- using this command you can unmount your volume.

- **After Running That all Commands in EC2-1 you need to detach volume that you created and attached with your first Instance EC2-1**
- **And create second EC2-2 instance and attach that volume you recently detached and follow below commands to mount and use or find your data that you stored in your volume**
  - sudo su - ---- go to root user
  - sudo mkdir /newdata ---- create directory to mount your data in that directory
  - sudo mount /dev/nvme1n1 /newdata ---- using this command mount your volume with that directory
  - cd /newdata ---- go to that directory
  - ls ---- to check your file and all data are available or not

--- if data is available your successfully done the practical of mount and format volume and access data ---

VIVEK TAWARE 9322662939

Term	Meaning	Purpose
EBS	Elastic Block Store	Persistent block storage for EC2
Volume	Virtual disk	Provides storage
Block Device	Disk exposed to Linux	Allows OS to interact with storage
`/dev/nvme1n1`	Linux device name	Represents your EBS disk
Filesystem	Structure used to store files	Makes disk usable for files
XFS	Linux filesystem	Used to organize/store files
`mkfs`	Make filesystem	Creates filesystem
Mount	Attach filesystem to directory	Makes storage accessible
Mount Point	Directory where disk is mounted	Example `/mydata`
`lsblk`	List block devices	Check attached disks
`fdisk -l`	List disk/partition information	Inspect disks
`file -s`	Identify filesystem/data	Check disk contents/type

What is an EBS volume?

---- An EBS (Elastic Block Store) Volume is basically a virtual hard disk that you can attach to an EC2 instance and detach

6. Create Snapshot of EBS Volume

Create EBS Volume → Attach to EC2 → Put data → Create Snapshot → Create new Volume from Snapshot → Attach new Volume to EC2 in the same Availability Zone → Mount and verify data

VIVEK TAWARE 9322662939

1. Check Your EC2 Availability Zone

- Go to – AWS Console → instances → select your EC2 instance
- Look for:
  - • Availability Zone: for Example: - ap-south-1a
  - • Instance ID
  - • Instance state: Running

  Important: An EBS volume must be created in the **same Availability Zone** match with the EC2 instance if you want to attach it directly.

2. Part 1 – create an EBS Volume

- Go to: AWS Console → Elastic Block Store → Volume
- Click: Create Volume (select or choose the specification that you want while creating the volume)

3. Part 2 – Attach Volume to EC2

- Select your newly Created volume
- Click on: actions → attach volume
- Select your EC2 instance that want to attach
- Example: -
  1. volume: - xxxxxxx
  2. Instance: - xxxxxx
  3. Device: - /dev/sdf

- Click on attach volume
- Now the volume state show: -- in use

4. Snapshot Creating ---

- create snapshot
- Go to AWS console → EC2 → Elastic Block Volume → Volumes
- select your original volume
- click on: Actions → create snapshot
- Enter Description ---- snapshot of original EBS Volume
- you can also add a tag
- Name – give name (EBS-snapshot-Test)
- click on create snapshot

7. Check Snapshot

- **EC2 → Elastic Block Store → Snapshots You should see: EBS-Snapshot-Test**
- is the snapshot status: pending (wait until status show's: completed)

8. Create a NEW Volume from Snapshot

- This is the important part of your practice
- EC2 → Elastic Block Store → Snapshots
- Select: EBS-Snapshot-Test
- Click: Actions → Create volume from snapshot
- Configure:

Setting	**Example**
Snapshot	Your snapshot
Volume type	gp3
Size	10 GiB or larger
Availability Zone	Same AZ as EC2
Encryption	Same/default
Tag	Name = Restored-EBS-Volume

Important: -

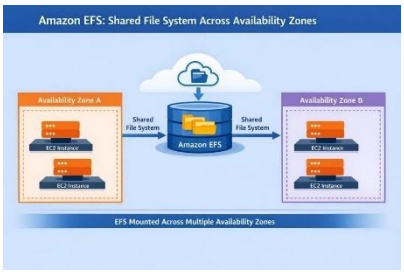
- If your EC2 is: ap-south-1a create the new volume in: ap-south-1a Then click: Create volume

9. Part 6 — Attach the New Volume

- Go to: EC2 → Volumes Find: Restored-EBS-Volume It should show: Available
- Select it: Actions → Attach volume Select your EC2.
- Example: Instance: xxxxxx Device: /dev/sdg
- Click: Attach volume
- Now: Restored-EBS-Volume State: In-use

VIVEK TAWARE 9322662939

❖ Elastic File System --- EFS creating EFS file and Mount it



- EFS is a Scalable cloud file storage service that allows multiple servers or applications to access the same file simultaneously
- EFS (Elastic File System) is an Aws Storage Service That allows Multiple servers to store share, and access the same file at the same Time
- Just like multiple people can access files in a shared folder, multiple AWS servers can access the same files using EFS

1. Step 1 — Create two EC2 instances

- Create: EC2-1
- OS: Ubuntu
- Availability Zone: ap-south-1a
- Security Group: Choose NFS

2. EC2-2

- OS: Ubuntu
- Availability Zone: ap-south-1b
- Security Group: Choose NFS

*** the availability zone chooses while making subnet and same instance choose for that availability zone They can be in same VPC

*Important Both EC2 instances need network connectivity to EFS. For this practical, using the same VPC makes things easier.

- ➢ Step -1 – Create EFS Elastic File system
- ➢ Go to: AWS console → Search EFS → Click on File System → Create File System
- ➢ Click : Create File System
- ➢ Select Your VPC

- For example: ---- VPC: Vivek-VPC -------Then create the file system
  - ➢ Step 3 -- Connect EC2-1
- **Follow Below Commands To install NFS in EC2 Server**
  1. sudo su → After connecting EC2 Go to Root User
  2. Sudo apt update
  3. Sudo apt install nfs-kernel-server -y → to install nfs in your server
  4. Sudo systemctl status nfs-kernel-server → to check status is active or not
  5. Sudo systemctl start nfs-kernel-serve → if not active use this command
  6. After install NFS in your server create a directory
  7. mkdir efs → this directory use to mount the EFS system
  8. after creating directory
  9. Go to: -- EFS
  10. Click on name of EFS

VIVEK TAWARE 9322662939

11. Go to Network ---- select manage
12. After Clicking on Manage ---- you see Mount Target Section
13. Choose Your Availability Zone Of Subnet That Match With Your EC2
14. Click on security groups ---- select your Instance (lunch wizard) ---- example(launch-wizard 7)
15. After select save all changes
16. And click on attach

- For now You Have IP address to attach EFS ---- when you have DNS you use Mount via DNS
- For now You Mount via IP
- You Attach Via IP You can Choose Your Availability Zone

17. And After That you see Mount Commands For Mount Paste That Commands In Your Server
18. For Example,

- 1) sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport 192.168.0.157:/ efs
- 2) sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport [$MOUNT_TARGET_IPv6]:/ efs

** After That Your EC2-1 Work Compete to Mount EFS Next Is You Can Access Same EFS with Another Instance and At the same Time Your Both Instance Work Stored In EFS **

- **To Access EFS on Second EC2 Server Follow Below Steps**

1. sudo su → After connecting EC2 Go to Root User
2. Sudo apt update
3. Sudo apt install nfs-kernel-server -y → to install nfs in your server
4. Sudo systemctl status nfs-kernel-server → to check status is active or not
5. Sudo systemctl start nfs-kernel-serve → if not active use this command
6. After install NFS in your server create a directory
7. mkdir efs → this directory use to mount the EFS system
8. after creating directory
9. Go to: -- EFS
10. Click on name of EFS
11. Go to Network ---- select manage
12. After Clicking on Manage ---- you see Mount Target Section
13. Choose Your Availability Zone Of Subnet That Match With Your EC2
14. Click on security groups ---- select your Instance (lunch wizard) ---- example(launch-wizard 7)
15. After select save all changes
16. And click on attach

- For now You Have IP address to attach EFS ---- when you have DNS you use Mount via DNS
- For now You Mount via IP
- You Attach Via IP You can Choose Your Availability Zone

19. And After That you see Mount Commands For Mount Paste That Commands In Your Server
20. For Example,

- 1) sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport 192.168.0.157:/ efs

17. 2) sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport [$MOUNT_TARGET_IPv6]:/ efs

VIVEK TAWARE 9322662939

1) RDS Database Creation And RDS to RDS data Transfer / Migration

- **Amazon RDS (Relational Database Service)** is a fully managed web service provided by Amazon Web Services (AWS) that simplifies the setup, operation, and scaling of relational databases in the cloud.
- **Follow Below Steps To Perform Operation of RDS to RDS database Transfer**
- To Transfer Data Create Two EC2-instances
- While Creating EC2 Instance Make Sure ---- in security Group -- allow the MysqlAurora Traffic
- Create Two EC2 Instance In same Availability Zone

  - **Steps To Create RDS database**
  - Open RDS Aurora – Search AWS Console
  - Open RDS –Click on Create Database
  - **Select:** Full Configuration While creating
  - **Engine Option / database:** select mysql
  - **DB Instance Size:** Fee Tier, if available for your account
  - **Credential Management:** Select – Self Managed
  - **Enter Password:** in Master Password section
  - **Click on set up EC2 connection:** click on --- connect to an EC2 Compute Resource
  - **Select Your EC2 instance:** if you don't have instance create instance
  - **Scroll down and create database**
  - **Connect your EC2 instance to server**
  - **After Connect Go to root user** sudo su –
- **And Follow This Commands**
  - Sudo apt update
  - Sudo apt install mysql-server -y
  - Sudo systemctl status mysql
  - Sudo systemctl enable mysql
- After install mysql Go to Your RDS database – click on database name – go connectivity and security
- Scroll down and you the section of connect steps --- endpoint – commands to connect RDS Database
  - curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
  - mysql -h database-1.ccz6gsie61ch.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem

    ** that command available in connectivity and security section
- Copy that commands from connectivity and security section
- And paste as it is in your EC2 server -- after pasting enter password of RDS database that you mentioned while creating RDS database and you go to your mysql RDS database

Now You Can Create MYSQL Database and Create tables using below commands

- Show Databases; -- to show databases you have
- Create database myemp; -- creating new database using this command (myemp - is database name)
- Use myemp; -- we using this database this go to in your that database
- Create table emp(id integer, name varchar(50)); -- using this we create a table with two columns
- Insert int emp values(1,'vivek'); -- added data in our table
- Select * from emp; -- show data of that table

VIVEK TAWARE 9322662939

- Commit; -- commited off changes using that command
- Exit --exit from your mysql database
- mysqldump -h database-2.ch8i4wmemkon.us-west-1.rds.amazonaws.com -u admin -p myemp > /home/ubuntu/dump1.sql -- this command used to create dump.sql file
- vi public.pem --create key pair file for push dump file to second EC2
- chmod 500 public.pem -- give this file execute permission
- scp -i /root/public.pem /home/ubuntu/dump1.sql ubunut@"paste second EC2 public IP":/home/ubuntu/ (this command push the dump file to second EC2 in home ubuntu and go to second EC2)

** Now we Transfer Data in RDS to RDS database using SCP -I command and create dump.sql file

- **To Transfer RDS to RDS data We need Two RDS databases and that second database that need attached with Second EC2 while creating**
- Follow Same steps on Second EC2 Instance to install mysql
  - Sudo su -
  - Sudo apt update
  - Sudo apt install mysql-server -y
  - Sudo systemctl status mysql
  - Sudo systemctl enable mysql

- After install mysql Go to Your RDS database – click on database name – go connectivity and security
- Scroll down and you the section of connect steps --- endpoint – commands to connect RDS Database
  - curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
  - mysql -h database-1.ccz6gsie61ch.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem

  - create database myemp; --- the database name same as previous database
  - Exit From mysql Database
  - grep -n "GTID_PURGED" /home/ubuntu/dump1.sql --- finds unwanted part from dump file
  - sed -i '24d' /home/ubuntu/dump1.sql --- that unwanted part at line no 24 and this removes it
  - mysql -h database-2.ch8i4wmemkon.us-west-1.rds.amazonaws.com -u admin -p myemp < /home/ubuntu/dump1.sql --- to execute dump1.sql file
  - mysql -h database-1.ccz6gsie61ch.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem

    - run this command to go mysql database after execution
    - and follow that below commands to see your dump1 file is executed or not
      1. show database;
      2. use myemp;
      3. select * from emp;
- if show your tables and al your data that created in previous database then your
- ** RDS to RDS data transferred and Migrated successfully

VIVEK TAWARE 9322662939

“Docker-Based Node.js Application Deployment with Amazon DynamoDB on AWS EC2”

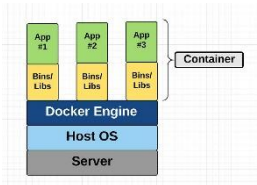
Docker is a platform used to build, package, and run applications inside containers.

In simple language, Docker packages an application together with the software and dependencies it needs, helping it run consistently on different systems.

Example: -

Suppose you develop a Node.js application on your laptop. It works perfectly, but when you run it on an Ubuntu EC2 instance, it fails because some dependencies or software versions are different.

Term	Simple meaning
Docker Image	A package containing the application and its requirements
Docker Container	A running instance of an image
Dockerfile	Instructions for building a Docker image
Docker Hub	An online registry for storing and sharing images
Docker Engine	The software that builds and runs containers
Docker Volume	Storage used to preserve data outside a container's writable layer
Docker Port Mapping	Connects a port on your EC2 instance to a port inside a container



VIVEK TAWARE 9322662939

# Check Docker version	docker basic commands
# Download an image	sudo docker pull nginx
# Run a container	sudo docker run -d --name myweb -p 80:80 nginx
# Check running containers	sudo docker ps
# Stop the container	sudo docker stop myweb
# Start the container again	sudo docker start myweb
# Remove the container	sudo docker rm -f myweb

Amazon DynamoDB is a fully managed NoSQL database service provided by AWS.

In simple language, it stores application data in the AWS cloud without requiring you to manage database servers yourself.

For example, your Node.js application could collect a user's name and email address from a website and save that information in DynamoDB.

How does DynamoDB store data?

DynamoDB organizes data into tables, items, and attributes.

Imagine a DynamoDB table called Contacts

id	username	email
101	Vivek	vivek@example.com
102	Rushi	rahul@example.com

Practical Docker Container and DynamoDB on EC2 Server

1. Create Public EC2 to perform this practical
2. Create DynamoDB Table follow below steps
3. Search DynamoDB on console – open DynamoDB
4. Open tables – Click on create table
5. Enter table name – (for practice purpose we the name as match with our docker hub repo 'Contacts')
6. Partition key – (we used 'id' as partition key same as table name because we docker hub repo)
7. Scroll Down and – click on Create table
8. Now Connect your EC2 with Server
9. After Connect EC2 – Go to IAM section or security credential – and create access for AWSCLI
10. And follow below commands to install Docker and AWSCLI

- sudo apt update #Step 1: Update Ubuntu
- sudo apt install awscli -y # step 2: install awscli
- sudo apt install ca-certificates curl -y #Step 2: Install required packages
- sudo install -m 0755 -d /etc/apt/keyrings #Step 3: Add Docker's official GPG key
- sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
  #step 4 : Docker's Offical Signing key
- sudo chmod a+r /etc/apt/keyrings/docker.asc #step 5 : Permission for docker

- echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
  #Step 6: Add the Docker repository #Copy and run this complete command

- sudo apt update #Step 7: Update package list
- sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
  #Step 8: Install Docker
- sudo systemctl start docker #Step 9: Start Docker
- sudo docker run hello-world #Step 9: Test Docker
- you paste all command in (.sh) file execute that file that makes you fast and execute that file (./docker.sh)

11. After That All commands Successfully Run And installed Docker continue to your practical

    We have a Docker Repo to test our Docker Image and our Database is working and storing data

VIVEK TAWARE 9322662939

12. Pull the repository in our Docker image we link that repo

- ---- https://hub.docker.com/r/philippaul/node-dynamodb-demo/#a-simple-node-based-webapp-which-takes-emailusername-from-frontend-and-store-into-dynamodb -----

13. After pull this repository --- configure AWS --- use below command to configure aws

- ---- aws configure # aws configure paste the secrete access key and access key

14. Aws configure give you access of DynamoDB

15. The run docker use the same that we have in that repository

- --- docker run --rm -d -p 80:3000 --name node-dynamo-app -e AWS_REGION=your-region -e AWS_ACCESS_KEY_ID=your-access-key -e AWS_SECRET_ACCESS_KEY=your-secret-key philippaul/node-dynamodb-demo ---

16. In that command you need change – your (AWS_REGION=" enter your DynamoDB region name")

17. Enter Access key (AWS_ACCESS_KEY_ID="Enter your access key")

18. Enter Your Secrete Access key (AWS_SECRETE_ACCESS_KEY="enter your access key")

19. That both keys you available in that tab of security credential tab or section if you downloaded that file you will get key in that excel file keys in that file

20. After that copy public IP of your EC2 server

21. Go to your browser and check your website is working if yes

22. Add usernames and save it

23. And then go to DynamoDB tab – open your table

24. By clicking table name – you will see the explore table section

25. Click on it and you will see the website data will store in our database table

- -- if you will see data in our table then the practical successfully completed ---
- ** common Error check (region while you running Docker using that command)
- **(also check table name that's very important)

** Install Docker → Pull image → Configure AWS/DynamoDB access → Run container → Open website **

VIVEK TAWARE 9322662939

AWS Cross-Account Access with IAM Role (S3 Read-Only)

Part 1: Share two things first

- Vivek → Rushikesh: Vivek's 12-digit AWS Account ID. He finds it by clicking his username at the top right of the console.
- Rushikesh → Vivek: Rushikesh's Account ID (found the same way), plus the role name he creates below.

Part 2: Rushikesh's steps (Account A)

Step 1: Create a bucket and upload a test file.
Go to S3 → Create bucket → upload any file (e.g., test.txt). Note the bucket name.

Step 2: Create the role.

1. Go to IAM → Roles → Create role.
2. Choose AWS account, then Another AWS account.
3. Enter Vivek's Account ID.
4. Click Next.

Step 3: Give S3 read-only permission.

1. In the search box, type AmazonS3ReadOnlyAccess.
2. Tick it and click Next.

For practice this is fine. It gives read access to all buckets in Rushikesh's account, so for real use a policy limited to one bucket is safer.

Step 4: Name the role.
Type the name CrossAccountS3ReadOnly and click Create role.

Step 5: Copy the role ARN and send it to Vivek.
Open the role you just created and copy the ARN. It looks like this:
arn:aws:iam::<RUSHIKESH_ACCOUNT_ID>:role/CrossAccountS3ReadOnly

Rushikesh's work is done.

Part 3: Vivek's steps (Account B)

Step 1: Allow your user to use the role.

1. Go to IAM → Users → vivek.
2. Click Add permissions → Create inline policy.
3. Open the JSON tab and paste this, replacing the Account ID with Rushikesh's:

json

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Action": "sts:AssumeRole",
   "Resource": "arn:aws:iam::<RUSHIKESH_ACCOUNT_ID>:role/CrossAccountS3ReadOnly"
  }
 ]
```

}

4. Click Next, name it AllowAssumeRushikeshRole, and click Create policy.

Step 2: Switch to the role.

1. Click your username at the top right.
2. Click Switch role.
3. Fill in:
    - Account: Rushikesh's Account ID
    - Role: CrossAccountS3ReadOnly
    - Display name: Rushikesh S3
4. Click Switch Role.

Step 3: Open the bucket.
Go to S3. The bucket list may look empty or show an error, because the role only has some permissions. Open the bucket by pasting this link in your browser, with the real bucket name:

https://s3.console.aws.amazon.com/s3/buckets/<bucket-name>

You should see test.txt and be able to open or download it.

Part 4: Test that it is really read-only

While switched into the role, try to upload or delete a file. It should fail with "Access Denied". That proves the role only gives read access.

If something goes wrong

Problem	Fix
"Switch role" fails	Check the Account ID and role name for typos.
Still fails	Rushikesh must have used Vivek's Account ID in Step 2. Vivek must have finish his Step 1.
Can see the bucket but not open files	If the files are encrypted with KMS, the role also needs kms:Decrypt.
Bucket list is empty	Normal. Open the bucket directly with its URL.

Both sides are needed: Rushikesh's role must trust Vivek's account, and Vivek's user must have permission to use that role. If one is missing, access is denied.

AMI and Load Balancer

What is an AMI?

An **AMI (Amazon Machine Image)** is a saved copy of a server. It holds the operating system, installed software, and settings. Instead of setting up a new server from scratch every time, you launch a new one from the AMI and it comes up already configured.

Example: You install a website on one server, save it as an AMI, and launch 10 identical servers from it in minutes.

Use cases of AMI

- **Quick copies:** launch many identical servers fast.
- **Backup:** keep a saved version of a working server in case something breaks.
- **Auto Scaling:** new servers added during heavy traffic start from the AMI.
- **Moving servers:** copy an AMI to another region to run the same server there.
- **Testing:** try changes on a copy without touching the real server.
- **Sharing:** give your AMI to another AWS account.

What is a Load Balancer?

A **Load Balancer** is a traffic manager. It sits in front of your servers and shares incoming visitors between them. It also checks whether each server is healthy and stops sending visitors to a server that is down.

Example: A shop with 3 billing counters and one person at the door sending each customer to the counter with the shortest line.

Use cases of Load Balancer

- **Handle high traffic:** no single server gets overloaded.
- **High availability:** if one server fails, the others keep the website running.
- **Health checks:** unhealthy servers are removed automatically.
- **No downtime updates:** update one server at a time while the others serve visitors.
- **One address:** users visit a single DNS name, no matter how many servers are behind it.
- **Works with Auto Scaling:** new servers are added to the load balancer automatically.

Steps to do the practical

1. **Launch one EC2 server** with a web page installed (use the user data script).
2. **Create an AMI** from that server (Actions → Image and templates → Create image).
3. **Wait** until the AMI status is **Available**.
4. **Launch 2 servers from the AMI**, each in a different Availability Zone.
5. **Create a Target Group** and add both servers.
6. **Create an Application Load Balancer** across two Availability Zones, forwarding to the target group.
7. **Open the load balancer's DNS name** in the browser and refresh. The page alternates between the two servers.
8. **Clean up** everything.

Do's

- **Do** wait for the AMI status to become **Available** before using it.
- **Do** give clear names (web-server-ami, web-tg, web-alb).
- **Do** put the two servers in **different Availability Zones**.
- **Do** allow **HTTP (port 80)** in the security group.
- **Do** check that targets show **Healthy** before testing.
- **Do** test by stopping one server to see the load balancer still working.
- **Do** delete all resources when finished.

Don'ts

- **Don't** launch servers from the AMI before it is Available.
- **Don't** put both servers in the same Availability Zone (you lose the high availability benefit).
- **Don't** forget the security group. A missing port 80 rule is the most common reason pages don't open.
- **Don't** open SSH (port 22) to everyone in a real project. It is fine for practice only.
- **Don't** store passwords or secret keys inside an AMI, because anyone who launches it gets them.
- **Don't** leave the load balancer, instances, AMI, or snapshots running after the practical. They keep costing money.
- **Don't** delete only the AMI. Also delete its **snapshot**, otherwise you are still charged for storage.
- **Don't** test the load balancer immediately. Wait until its state is **Active**.

Quick comparison

	AMI	**Load Balancer**
Purpose	Copies a server	Shares traffic between servers
Solves	Setting up many servers quickly	One server being overloaded or failing
Works with	EC2	EC2, Auto Scaling, Target Groups